

FastAPI Proje Refactoring ve Dockerizasyon

NTP II – Hafta 14



FastAPI Proje Refactoring ve Dockerizasyon

Bu ders, büyük ve sürdürülebilir API projeleri inşa etmenin temellerine odaklanacaktır.

Temel amacımız, 'spagetti kod'dan kurtulmak ve projeyi üretim ortamına hazırlamaktır.

Nesne Tabanlı Programlama (OOP) ilkelerinin, sadece sınıf yazmak değil, aynı zamanda modüler sistemler tasarlamak anlamına geldiğini göreceğiz.



Modül Öğrenme Hedefleri

Tüm kodun tek bir main.py dosyasında olmasının neden kötü bir pratik olduğunu anlamak (Sürdürülebilirlik, Ekip Çalışması, Spaghetti Code).
'Sorumlulukların Ayrılması' (Separation of Concerns) ilkesini öğrenmek.
FastAPI'nin APIRouter özelliğini kullanarak projeyi modüler dosyalara ayırmayı öğrenmek (Refactoring).
Üretim (production) ortamı ile geliştirme (development) ortamı arasındaki temel farkları anlamak.
Projeyi Dockerfile ile paketlemenin (containerization) ne anlama geldiğini kavramak.

Senaryo: Müşteri Talebi ve Problemin Tanımı

Müşteri, projenin harika ilerlediğini ancak main.py dosyasının yüzlerce satır uzunluğunda ve tam bir 'spagetti' haline geldiğini belirtiyor. Yeni bir geliştiricinin ekibe katıldığında bu monolitik yapıyı anlamasının günler alacağı endişesi var. Bu durum, projenin sürdürülebilirliğini ve ekip çalışmasını doğrudan tehdit ediyor.



Spagetti Kod Nedir ve Neden Kötüdür?

Tek bir dosyanın tüm iş mantığını, veritabanı işlemlerini, rotaları ve doğrulama şemalarını içermesi durumudur.

Okunabilirlik ve Anlaşılabilirlik: Yeni başlayanlar için kabus.

Hata Ayıklama (Debugging): Bir hatanın nerede olduğunu bulmak zordur.

Sürdürülebilirlik: Kod tabanı büyüdükçe değişiklik yapmak veya yeni özellik eklemek riskli hale gelir.

Ekip Çalışması: Aynı dosya üzerinde birden fazla geliştiricinin çalışması merge çakışmalarına yol açar.



Profesyonel Çözüm: Refactoring (Yeniden Düzenleme)

Refactoring, mevcut kodun dış davranışını değiştirmeden iç yapısını iyileştirme sürecidir.

Görevimiz, projeyi mantıksal olarak yeniden düzenlemektir:

Veritabanı mantığı (crud.py)

Modeller (models.py)

Veri Şemaları (schemas.py)

Kimlik Doğrulama (auth.py) için ayrı dosyalar/klasörler oluşturulmalıdır.

Temel İlke: Sorumlulukların Ayrılması (Separation of Concerns - SoC)

SoC, bir bilgisayar programını, her bir bölümün ayrı bir 'sorumluluğu' veya işlevi ele aldığı farklı bölümlere ayırma prensibidir.

Her modül, sadece kendisine ait olan görevleri yerine getirmelidir (Örn: Veritabanı mantığı sadece veritabanıyla ilgilenmelidir).

Bu, OOP'nin 'Modülerlik' ilkesiyle doğrudan ilişkilidir ve sürdürülebilir sistemler için kritik öneme sahiptir.



SoC Örneği: MVC Mimarisi

Model-View-Controller (MVC) gibi mimariler, SoC ilkesinin popüler uygulamalarıdır.

Model: Veri ve iş mantığı (Veritabanı işlemleri).

View: Kullanıcı arayüzü (Front-end).

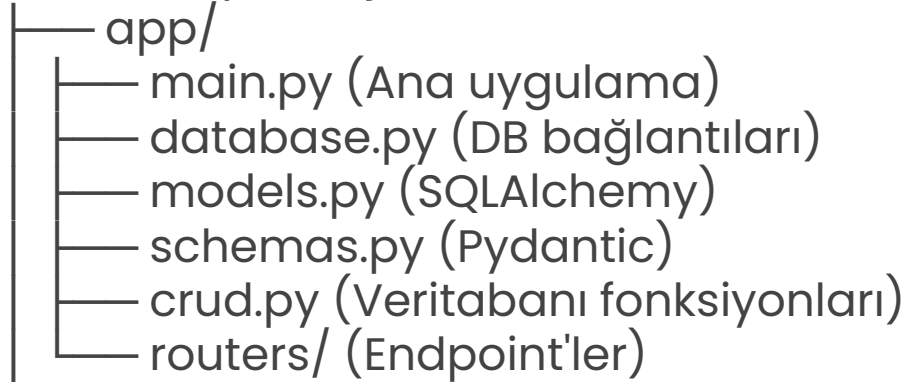
Controller: Model ve View arasındaki trafiği yöneten katman (API Rotaları).

FastAPI projemizde bu sorumlulukları dosya düzeyinde uygulayacağız.



Profesyonel FastAPI Proje Yapısı (Hedef)

Hedefimiz, ölçeklenebilir ve anlaşılır bir yapı kurmaktır:
/threat_api (Proje Kök Klasörü)



app/ Klasörünün İçeriği

app/ klasörü, uygulamanın temel çalışma mantığını barındırır.

`__init__.py`: Python'a bu klasörün bir paket olduğunu belirtir (Genellikle boş kalır).

`main.py`: Sadece ana FastAPI uygulamasını başlatır ve diğer modülleri/router'ları dahil eder.

`database.py`: SQLAlchemy engine, SessionLocal ve Base nesnelerini tanımlar.

models.py: Veritabanı Modelleri

Bu dosya, veritabanındaki tabloları temsil eden SQLAlchemy sınıflarını (ORM Modelleri) içerir.

Her sınıf, bir tabloyu ve bu tablonun sütunlarını tanımlar.

Örnek: User modeli, id, email, hashed_password gibi alanları tanımlar.

Bu modeller, veritabanı katmanından sorumlu olduğu için SoC ilkesine uyar.

schemas.py: Veri Şemaları

Pydantic, veri doğrulaması ve serileştirme için kullanılır. schemas.py, API giriş ve çıkış verilerinin yapısını tanımlayan Pydantic sınıflarını içerir. Bu, API'nin beklediği ve döndürdüğü veri formatlarının net olmasını sağlar. Örnek: UserCreate (API'dan gelen giriş verisi) ve User (API'nin döndürdüğü çıkış verisi).

crud.py: Veritabanı İşlemleri

Bu dosya sadece veritabanı ile etkileşime giren fonksiyonları içerir. crud, veritabanı oturumunu (db session) girdi olarak alır ve kullanıcı oluşturma, veri okuma veya güncelleme gibi işlemleri yapar. API rotaları, veritabanı mantığını değil, bu crud fonksiyonlarını çağırır.

auth.py: Kimlik Doğrulama ve Güvenlik

Kimlik doğrulama (Authentication) ve yetkilendirme (Authorization) ile ilgili tüm mantık buraya taşınır.

Kullanıcı şifrelerinin hashlenmesi, JWT (JSON Web Token) oluşturulması ve token doğrulama bağımlılıkları bu dosyada yer alır.



Refactoring Aracı: FastAPI APIRouter

APIRouter, FastAPI'nin büyük uygulamaları yönetmek için tasarlanmış temel özelliğidir.

Her bir mantıksal grup (örneğin 'Kullanıcılar' veya 'Göstergeler') için ayrı bir router nesnesi oluşturulabilir.

Bu router'lar bağımsız olarak çalışır ve daha sonra ana uygulamaya dahil edilir (`include_router`).

APIRouter Uygulaması: routers/ Klasörü

Tüm endpoint fonksiyonlarını içeren dosyalar, app/routers/ klasörü içine taşınır.

Örnek olarak indicators.py ve users.py dosyaları oluşturulur.

Her dosya kendi APIRouter örneğini tanımlar ve ilgili endpoint'leri bu router'a ekler.



Örnek: routers/indicators.py

Bu dosya, sadece /indicators ile ilgili rotaları barındırır.
Öncelikle bir APIRouter nesnesi tanımlanır.
Ardından, main.py'den kesilen tüm /indicators endpoint'leri buraya yapıştırılır.
Veritabanı bağımlılıkları (örn. Depends(get_db)) buraya uygun şekilde güncellenir.

Örnek: routers/users.py

Bu dosya, /users/signup ve /token endpoint'lerini içerir.
Kullanıcı kaydı ve kimlik doğrulama işlemleri burada ele alınır.
Bu sayede, kimlik doğrulama mantığı (auth.py) ve rota mantığı (users.py) ayrılmış olur, ancak birbirini kullanır.



main.py'nin Yeni Rolü

Refactoring sonrası main.py, yüzlerce satırlık iş mantığından arındırılır. Artık sadece şu görevleri yerine getirir:

1. FastAPI uygulamasını başlatmak.
2. Veritabanı tablolarını oluşturmak (Base.metadata.create_all).
3. Middleware'leri (örn. CORS) eklemek.
4. APIRouter modüllerini ana uygulamaya dahil etmek ('include' etmek).

main.py Kodu: Router Dahil Etme

Router'ları ana uygulamaya dahil etmek için `app.include_router()` kullanılır.

```
app.include_router(users.router)
app.include_router(indicators.router, prefix=/indicators, tags=[Indicators])
```

'prefix', rotaların önüne eklenecek yol önekini belirler.

'tags', otomatik OpenAPI (Swagger) dokümantasyonunda grupta sağlar.

Middleware Yönetimi

CORS (Cross-Origin Resource Sharing) ayarları gibi uygulama genelindeki yapılandırmalar middleware olarak tanımlanır.

Bu ayarların da main.py'den ayrı bir dosyaya (middleware.py) taşınması profesyonel bir yaklaşımdır.

Temizlenmiş main.py, sadece bu middleware'i çağırır:
`add_cors_middleware(app).`

Sağlık Kontrolü (Health Check)

Tüm bu refactoring sürecinin ardından, uygulamanın doğru çalıştığını test etmek için basit bir '/health' endpoint'i bırakılabilir. Bu endpoint genellikle {'status': 'ok'} döndürür ve dağıtım ortamlarında uygulamanın ayakta olup olmadığını kontrol etmek için kullanılır.

OOP ve Modülerlik İlişkisi

Nesne Tabanlı Programlamanın sadece sınıf (class) tanımlamak olmadığını anlamak önemlidir.

Modülerlik, sistemin farklı parçalarının birbirinden bağımsız çalışabilmesini ve ayrı ayrı geliştirilebilmesini sağlar.

Dosyaları ayırmak, her modülün net bir sorumluluğa sahip olduğu 'Yüksek Uyum' (High Cohesion) ve 'Düşük Bağımlılık' (Low Coupling) ilkelerini destekler.

Kısa Özet: Refactoringin Faydaları

Sürdürülebilirlik: Kodun ömrünü uzatır.

Test Edilebilirlik: Küçük, izole edilmiş modüllerin test edilmesi çok daha kolaydır.

Geliştirici Verimliliği: Yeni özellikler, mevcut kodu bozma korkusu olmadan eklenebilir.

Ekip Ölçeklenebilirliği: Farklı ekipler farklı modüller üzerinde eş zamanlı çalışabilir.



Bölüm Sonu: Refactoringin Sonuçları

7 hafta önce 10 satırlık basit bir main.py ile başladık.
Şimdi ise 10'dan fazla dosyadan oluşan, modüler, güvenli ve katmanlı bir projeye sahibiz.
Bu, bir üniversite projesinden profesyonel bir tehdit istihbaratı API'sine geçiş demektir.



Üretim ve Geliştirme Ortamları

Geliştirme (Development): Geliştiricinin kendi makinesi, sürekli değişen kod, hata ayıklama modu, sahte veya yerel veritabanı kullanılır.

Üretim (Production): Gerçek kullanıcıların eriştiği canlı ortam. Güvenilirlik, performans, güvenlik ve kararlılık önceliklidir.

Kodun her iki ortamda da aynı şekilde çalışmasını sağlamak kritik bir zorluktur.

Dağıtım (Deployment) Zorlukları

Bu, yazılım geliştirmede en yaygın sorundur.
Sorunun temel nedeni, bir uygulamanın bağımlılıklarının (kullanılan Python kütüphaneleri, işletim sistemi versiyonları, ortam değişkenleri) geliştirme ve üretim ortamlarında farklı olmasıdır.
Bu bağımlılıkları standartlaştırmamız gerekiyor.



Çözüm: Konteynerleştirme (Containerization)

Konteynerleştirme, uygulamayı ve tüm bağımlılıklarını, ayarlarını ve kütüphanelerini tek bir izole pakete (konteyner) sarmalama sürecidir. Bu konteyner, çalıştırıldığı her yerde aynı ortamı garantiler. Docker, bu konteynerleri oluşturmak ve yönetmek için kullanılan en popüler araçtır.

Docker Konteyner Nedir?

Konteynerler, Geleneksel Sanal Makinelardan (VM) farklıdır: VM'ler tüm işletim sistemini taklit eder (ağır). Konteynerler, ana makinenin işletim sistemi çekirdeğini paylaşır ve sadece uygulama seviyesinde izolasyon sağlar (hafif ve hızlı).

Dockerfile Nedir?

Dockerfile, bir Docker imajını (kalıp) adım adım inşa etmek için gereken talimatları içeren metin tabanlı bir dosyadır.

Bu talimatlar şunları içerir:

FROM: Hangi temel imajı kullanacağımız (Örn: Python 3.10)

COPY: Kodumuzu veya bağımlılık listemizi kopyalamak

RUN: Komutları çalıştırmak (Örn: requirements.txt'deki kütüphaneleri kurmak)

CMD: Konteyner başlatıldığında çalışacak ana komut (Örn: uvicorn).

Dockerfile Adım 1: Temel İmajı Seçme

Tüm Dockerfile'lar, bir temel imajla başlar.

FastAPI projemiz için en uygunu resmi bir Python imajıdır (Örn: FROM python:3.10-slim).

'slim' versiyonları, gereksiz dosyaları içermediği için daha küçük ve güvenlidir.



Dockerfile Adım 2: Bağımlılıkların Tanımlanması

Uygulamamızın tüm bağımlılıklarını requirements.txt dosyasına listelememiz gerekir.

Dockerfile'da bu dosyayı kopyalarız:

`COPY requirements.txt .`

Ardından, bu bağımlılıkları sanal ortam içinde kurarız:

`RUN pip install --no-cache-dir -r requirements.txt.`

Dockerfile Adım 3: Çalışma Dizini ve Kodu Kopyalama

WORKDIR /app komutu, konteyner içindeki tüm sonraki komutların çalışacağı dizini ayarlar.

COPY . /app komutu, yerel proje klasörümüzdeki tüm kodları (app/, Dockerfile hariç) konteyner içindeki /app dizinine kopyalar.

Dockerfile Adım 4: Başlangıç Komutu

CMD veya ENTRYPOINT, konteyner başlatıldığında çalışacak ana komuttur. FastAPI uygulamaları genellikle Uvicorn ile çalıştırılır (ASGI sunucusu)



Docker Compose Nedir?

Docker Compose, birden fazla konteyneri (servis) tanımlamak ve çalıştırmak için bir araçtır.

Karmaşık uygulamalar genellikle birden fazla servis gerektirir: API, Veritabanı (PostgreSQL/MongoDB), Önbellek (Redis).

docker-compose.yml dosyası, bu servislerin yapılandırmasını (ağlar, portlar, bağımlılıklar) YAML formatında tutar.

docker-compose.yml Yapısı (API Servisi)

API servisi tanımı, projeyi derleyeceği (build: .) ve hangi portları açacağını (ports: '8000:8000') belirtir.

volumes: Geliştirme sırasında kod değişikliklerinin konteynere anında yansımalarını sağlayan yerel dizin eşlemesi için kullanılabilir.

docker-compose.yml Yapısı (Veritabanı Servisi)

Compose, PostgreSQL gibi bağımsız bir veritabanı servisini kolayca ayağa kaldırmamızı sağlar.

Bu, API'mizin veritabanına olan bağımlılığını basitleştirir.

Ortam değişkenleri (Environment variables) ile DB şifreleri ve bağlantı bilgileri bu servis içinde güvenli bir şekilde tanımlanır.



Tek Komutla Dağıtım: docker-compose up

docker-compose up komutu, docker-compose.yml dosyasındaki tüm servisleri okur, gerekli imajları (FastAPI ve DB) indirir/derler ve tek seferde tüm konteynerleri başlatır.

Bu, projenin geliştiricinin makinesinde veya bir üretim sunucusunda tutarlı bir şekilde çalışmasını garanti eden 'Bonus' görevimizin çözümüdür.



Ortam Değişkenlerinin Yönetimi

Gizli bilgilerin (API anahtarları, DB şifreleri) koda gömülmemesi esastır. Ortam değişkenleri (Environment Variables) bu bilgileri çalıştırma sırasında dinamik olarak sağlamamızı sağlar. Docker ve Docker Compose, .env dosyaları veya doğrudan Compose dosyası aracılığıyla ortam değişkenlerini kolayca yönetir.



Refactoringin Temel Etkisi

Bu hafta öğrendiklerimiz, sadece kod yazma becerisini değil, aynı zamanda yazılım mühendisliği disiplini de kapsar.

Profesyonel bir yazılım geliştiricisi, kodu çalışır hale getirmekten öte, kodun yarın da çalışabilir ve değiştirilebilir olmasını sağlamalıdır.

Modüler yapı, gelecekte Microservices mimarisine geçiş için de bir temel oluşturur.



Sürdürülebilir Geliştirme Döngüsü

Modüler kod (Refactoring) geliştirme hızını artırır.
Konteynerizasyon (Docker), dağıtım hızını ve güvenilirliğini artırır.
Bu iki kavramın birleşimi, modern DevOps (Geliştirme ve Operasyon) pratiğinin temelini oluşturur.



Akademik Bağlam: SOLID Prensipleri

Sorumlulukların Ayrılması (SoC), SOLID prensiplerinden 'Tek Sorumluluk Prensipleri' (Single Responsibility Principle - SRP) ile yakından ilişkilidir. SRP: Bir sınıfın veya modülün değişmesi için yalnızca tek bir nedeni olmalıdır. crud.py (DB işlemleri) veya schemas.py (Veri doğrulama), SRP'ye uyan modüllere mükemmel birer örnektir.

Uygulama Hızı: ASGI Sunucuları

FastAPI, yüksek performans için asenkron bir web sunucusu (ASGI) olan Uvicorn'u kullanır.

Docker konteyneri içinde Uvicorn'un doğru şekilde yapılandırılması, üretim ortamında API'nin maksimum verimlilikle çalışmasını sağlar.

CMD komutundaki 0.0.0.0 host tanımı, konteyner dışından erişimi mümkün kılar.



Proje Yönetimi ve Sürüm Kontrolü

Projeyi modüler dosyalara ayırmak, Git gibi sürüm kontrol sistemleriyle çalışmayı kolaylaştırır.

Farklı ekipler farklı dosyalarda (indicators.py vs. users.py) çalıştığı için merge çakışmaları azalır.

Her dosyanın sorumluluğu net olduğu için kod incelemesi (Code Review) süreçleri hızlanır.



Sonuç ve İlerideki Adımlar

Artık sadece çalışan değil, aynı zamanda iyi tasarlanmış, modüler ve tek bir komutla dağıtılabilen bir API projemiz var.

Bu yapı, daha büyük kurumsal sistemlere ve mikroservis mimarilerine geçiş için sağlam bir temel sunar.

Bu ders, 'Nesne Tabanlı Programlama' dersinin temel felsefesini somutlaştırmıştır: Tasarım önemlidir.



FastAPI Proje Refactoring ve Dockerizasyon

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

